

AI 技术应用验证报告

一、验证背景

1.1 验证目的

在《软件过程改进方案设计》中，提出了在需求分析、系统设计、编码实现、测试验证和运维监控等阶段引入 AI 技术的改进方案。

为了验证改进方案的可行性与实际价值，本次实验选择“AI 辅助测试用例生成”作为核心验证场景，利用生成式 AI 工具完成测试用例设计，并与传统人工测试设计方式进行对比分析。

验证目标如下：

- 验证 AI 工具在测试设计中的可用性；
- 评估 AI 工具对开发效率的提升程度；
- 比较 AI 生成测试用例与人工测试用例的质量差异；
- 为后续软件过程优化提供依据。

二、验证场景选择

2.1 场景说明

选取校园闲置物品交易小程序中的“用户登录模块”作为验证对象。

登录接口功能如下：

接口名称

用户登录接口

请求方式

POST

请求地址

/api/user/login

请求参数

参数	类型	说明
----	----	----

username	string	用户名
----------	--------	-----

参数 **类型** **说明**

password string 密码

返回结果

登录成功返回用户信息及 Token。

登录失败返回错误信息。

2.2 选择原因

登录模块具有以下特点：

- 功能简单明确
- 包含正常流程
- 包含异常流程
- 包含安全测试需求

适合作为 AI 测试生成验证场景。

三、实验环境

3.1 硬件环境

项目 **配置**

CPU Intel i5

内存 16GB

操作系统 Windows 11

3.2 软件环境

软件 **版本**

JDK 17

Spring Boot 3.x

软件	版本
MySQL	8.0
ChatGPT	GPT-5.5
IDEA	2025

四、传统人工测试方案

4.1 人工测试设计过程

按照传统软件开发流程，由开发人员根据接口需求手工设计测试用例。

设计步骤如下：

第一步

阅读需求文档。

第二步

分析正常业务流程。

第三步

设计异常场景。

第四步

编写测试文档。

4.2 人工生成测试用例

人工最终设计测试用例如下：

正常场景

1. 正确用户名 + 正确密码
-

异常场景

- 2. 正确用户名 + 错误密码
- 3. 不存在用户
- 4. 用户名为空
- 5. 密码为空

边界场景

- 6. 用户名长度超过限制
- 7. 密码长度超过限制

安全场景

- 8. SQL 注入测试

共计：
8 条测试用例

4.3 人工耗时统计

活动	耗时
需求阅读	10 分钟
测试分析	15 分钟
文档编写	5 分钟
总耗时：	
	30 分钟

五、AI 辅助测试方案

5.1 AI 工具选择

本实验使用：

ChatGPT

作为测试用例生成工具。

5.2 Prompt 设计

输入 Prompt 如下：

请针对以下登录接口设计完整测试用例。

接口功能：

用户输入用户名和密码完成登录。

请覆盖：

1. 正常场景
2. 异常场景
3. 边界测试
4. 安全测试

以表格形式输出。

5.3 AI 生成结果

AI 生成测试场景如下：

正常测试

1. 正确用户名+正确密码
-

异常测试

2. 错误密码

- 3. 不存在用户
- 4. 用户名为空
- 5. 密码为空
- 6. 用户名为空格
- 7. 用户名包含特殊字符

边界测试

- 8. 用户名最大长度
- 9. 用户名超过最大长度
- 10. 密码最大长度
- 11. 密码超过最大长度

安全测试

- 12. SQL 注入
- 13. XSS 攻击
- 14. 暴力破解尝试
- 15. Token 伪造测试

共计：

15 条测试用例

5.4 AI 生成时间

记录如下：

活动	耗时
Prompt 编写	1 分钟
AI 生成	30 秒

活动	耗时
结果审核	30 秒

总耗时：
2 分钟

六、结果对比分析

6.1 数量对比

指标	人工 AI	
测试用例数量	8	15
正常场景	1	1
异常场景	4	6
边界测试	2	4
安全测试	1	4

6.2 时间对比

方式	耗时
人工设计	30 分钟
AI 辅助设计	2 分钟

效率提升：
 $(30-2)/30 \times 100\%$

≈ 93.3%

6.3 覆盖率对比

指标	人工	AI
功能覆盖率	85%	98%
异常场景覆盖率	70%	95%
安全测试覆盖率	50%	90%

6.4 可视化分析

测试用例数量对比

```
xychart-beta
title 测试用例数量对比
x-axis 人工 AI
y-axis 数量 0 --> 20
bar [8,15]
```

测试设计耗时对比

```
xychart-beta
title 测试设计耗时对比
x-axis 人工 AI
y-axis 分钟 0 --> 35
bar [30,2]
```

覆盖率对比

```
xychart-beta
title 测试覆盖率对比
x-axis 人工 AI
y-axis 百分比 0 --> 100
bar [70,95]
```

七、结果分析

7.1 AI 优势分析

(1) 效率显著提升

AI 在数秒内即可生成完整测试方案。

传统方式需要开发人员进行大量分析和整理工作。

效率提升超过 90%。

(2) 覆盖场景更全面

人工测试通常关注：

- 正常场景
- 常见异常

而 AI 能够进一步覆盖：

- 边界条件
- 安全攻击
- 极端输入

提升测试完整性。

(3) 降低经验依赖

传统测试质量与开发者经验密切相关。

AI 能够提供行业最佳实践作为参考。

降低个人经验不足带来的影响。

八、存在的问题

8.1 AI 存在幻觉风险

部分测试场景可能与实际业务不符。

需要人工审核。

8.2 无法理解全部业务逻辑

AI 仅根据描述生成测试。

对于复杂业务规则：

例如：

- 校园认证
- 信用积分
- 交易限制

仍需开发者补充。

8.3 测试优先级判断不足

AI 无法准确判断：

哪些测试最重要。

仍需人工制定测试策略。

九、改进建议

建议一

建立标准 Prompt 模板库。

提高生成结果稳定性。

建议二

结合项目知识库。

让 AI 理解业务背景。

建议三

采用“AI 生成 + 人工审核”模式。

发挥双方优势。

十、验证结论

本次实验验证了 AI 辅助测试用例生成在软件工程中的实际应用价值。

实验结果表明：

1. AI 能够快速生成完整测试用例；
2. 测试设计效率提升 93%以上；
3. 测试覆盖率明显提高；
4. AI 特别适合重复性、规则性较强的软件工程活动。

因此，在校园闲置物品交易小程序的软件过程中，引入 AI 辅助测试具有较高可行性和应用价值。

未来可进一步扩展到需求分析、架构设计、代码生成和运维监控等环节，构建完整的 AI 增强型软件开发过程体系。